JAVASCRIPT · TEMA 6 · GUÍA DE APOYO AL ALUMNADO

Eventos, formularios y validación en JavaScript

Cómo hacer que una página responda a las acciones del usuario. Con explicaciones, ejemplos de código y actividades progresivas.

eventos · propagación · formularios · validación · delegación

01 ¿Qué es un evento?

Un **evento** es una señal de que algo ha ocurrido: se ha pulsado un botón, ha cambiado un campo, se ha enviado un formulario o ha terminado un vídeo. El navegador proporciona esa señal y JavaScript permite decidir qué hacer al recibirla.

Acción del usuario → **Evento** → **Función manejadora** → **Cambio en la interfaz**

Ejemplo mental

Un botón «Añadir» no sabe guardar productos por sí solo. Al registrar un manejador para su evento `click`, indicamos qué función debe ejecutar el navegador cuando se active.

Hasta ahora podías ejecutar instrucciones en orden y modificar el DOM. Con los eventos, parte del código queda preparada para ejecutarse **cuando suceda algo**. Registrar una función y ejecutarla son dos acciones distintas.

Al terminar esta guía podrás…

- Registrar y retirar manejadores de eventos.
- Leer y validar formularios con HTML y JavaScript.
- Interpretar el objeto del evento y su recorrido por el DOM.
- Gestionar elementos dinámicos mediante delegación.

Punto de partida: funciones, objetos, `querySelector`, `textContent` y creación de elementos. Cada ejemplo pequeño se prueba por separado; el catálogo de los apartados finales sí forma un único proyecto.

02 Preparar el HTML y escuchar un clic

Crea una carpeta con `index.html` y `app.js`. El HTML define los elementos; JavaScript define su comportamiento. Abre el HTML en el navegador y la consola de las herramientas de desarrollo.

HTML · index.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Primer evento</title>
  <script src="app.js" defer></script>
</head>
<body>
  <button id="saludar" type="button">Saludar</button>
  <p id="mensaje" role="status"></p>
</body>
</html>
```

¿Para qué sirve defer?

En un script externo clásico, `defer` permite descargar el archivo sin detener el análisis del HTML y ejecutarlo después de que este se haya analizado. Así, los elementos ya existen cuando los buscamos.

JavaScript · app.js

```javascript
const boton = document.querySelector("#saludar");
const mensaje = document.querySelector("#mensaje");

function saludar() {
  mensaje.textContent = "¡Hola, DAW!";
}

boton.addEventListener("click", saludar);
```

1. `querySelector` obtiene el botón y el párrafo.
2. `saludar` contiene la respuesta que queremos ejecutar.
3. `addEventListener` registra la función para el evento `click`.
4. Al activar el botón, el navegador llama a `saludar` y cambia el texto.

Error típico: ejecutar en lugar de registrar

Escribe `saludar`, sin paréntesis. `saludar()` ejecutaría la función en ese instante y pasaría su resultado como manejador. Aquí ese resultado sería `undefined`.

Prueba también con el teclado

Usa Tab para enfocar el botón y pulsa Enter o la barra espaciadora. Un `<button>` nativo ya tiene comportamiento de teclado; no hace falta reconstruirlo con un `<div>`.

03 addEventListener y removeEventListener

Un **listener** es una escucha registrada. Su **manejador** es la función que se ejecuta cuando llega el evento. Puede ser una función con nombre o una función flecha.

JavaScript · dos formas de registrar

```
function mostrarTipo(evento) {
  console.log(evento.type);
}

boton.addEventListener("click", mostrarTipo);

boton.addEventListener("click", () => {
  console.log("Otra respuesta al mismo clic");
});
```

Forma	Qué debes saber
`onclick="…"` en HTML	Mezcla estructura y comportamiento. Evítalo como forma habitual de trabajo.
`boton.onclick = funcion`	La propiedad guarda un manejador; otra asignación a esa propiedad sustituye el anterior.
`addEventListener`	Permite varias escuchas para el mismo evento y configurar sus opciones.

Retirar una escucha

Hay que conservar la **misma referencia a la función**. Dos funciones flecha escritas igual son dos objetos distintos.

JavaScript · misma función al añadir y al quitar

```
function avisar() {
  console.log("Se ha pulsado el botón");
}

boton.addEventListener("click", avisar);
boton.removeEventListener("click", avisar);
// Al pulsar, avisar ya no se ejecutará.

// Esto NO retira la primera función flecha:
boton.addEventListener("click", () => console.log("Hola"));
boton.removeEventListener("click", () => console.log("Hola"));
```

Opción	Uso
`{ once: true }`	Retira automáticamente la escucha después de su primera ejecución.
`{ capture: true }`	Escucha durante la captura; se explica en el apartado 05.
`{ passive: true }`	Declara que el manejador no cancelará la acción predeterminada. No llames a preventDefault en él.

JavaScript · ejecutar una única vez

```
boton.addEventListener("click", avisar, { once: true });
```

Tres datos deben coincidir al retirar

El elemento, el tipo de evento y la función deben ser los mismos. Además, el valor de `capture` debe coincidir. Las otras opciones no intervienen en esa coincidencia.

04 El objeto del evento

El navegador entrega al manejador un objeto con información sobre lo ocurrido. Puedes llamar al parámetro `evento`, `event` o `e`: el nombre lo eliges tú.

Propiedad	Qué representa
`type`	El nombre del evento, por ejemplo "click".
`target`	El objetivo donde se originó el evento en estos ejemplos de DOM.
`currentTarget`	El elemento cuya escucha se está ejecutando en ese momento.
`bubbles`	Si el evento puede recorrer los antecesores en burbujeo.
`cancelable`	Si admite cancelar su acción predeterminada.
`defaultPrevented`	Si la acción predeterminada ya se ha cancelado.

target y currentTarget no siempre coinciden

Supón que el botón contiene un `span` y pulsas exactamente sobre el texto de ese `span`.

HTML · botón con un elemento interior

```html
<button id="guardar" type="button">
  <span>Guardar producto</span>
</button>
```

JavaScript · distinguir el origen de la escucha

```javascript
const guardar = document.querySelector("#guardar");

guardar.addEventListener("click", (evento) => {
  console.log(evento.target.tagName);        // SPAN
  console.log(evento.currentTarget.tagName); // BUTTON
});
```

Lectura rápida

`target`: «¿dónde ocurrió?». `currentTarget`: «¿dónde estoy escuchándolo?». Si pulsas el fondo del botón, ambos pueden ser `BUTTON`.

Relación con this

En un manejador declarado con `function` y registrado directamente con `addEventListener`, `this` coincide con `currentTarget`. Una función flecha conserva el `this` de su contexto exterior. Para reconocer el elemento de la escucha, usa explícitamente `evento.currentTarget`.

Cada familia aporta información distinta

Un evento de teclado incorpora `key`; uno de ratón incorpora coordenadas. No todas las propiedades existen en todos los eventos. Además, `currentTarget` solo es útil durante la ejecución del manejador.

05 Captura, objetivo y burbujeo

Un elemento suele estar dentro de otros. Al hacer clic en un botón, el evento recorre esa estructura. Si no se detiene, las escuchas se ejecutan según su **fase**.

1. Captura

Por los antecesores, desde el exterior hacia el objetivo.

2. Objetivo

Se ejecutan las escuchas del elemento donde ocurre el evento.

3. Burbujeo

Si el evento burbujea, vuelve por los antecesores hacia el exterior.

HTML · elementos anidados

```html
<div id="zona">
  <button id="accion" type="button">Pulsar</button>
</div>
```

JavaScript · laboratorio de propagación

```javascript
const zona = document.querySelector("#zona");
const accion = document.querySelector("#accion");

document.body.addEventListener("click", () => {
  console.log("1. body: captura");
}, { capture: true });

zona.addEventListener("click", () => {
  console.log("2. zona: captura");
}, { capture: true });

accion.addEventListener("click", () => {
  console.log("3. botón: objetivo");
});

zona.addEventListener("click", () => {
  console.log("4. zona: burbujeo");
});

document.body.addEventListener("click", () => {
  console.log("5. body: burbujeo");
});
```

Resultado al pulsar el botón

Los mensajes aparecen en el orden **1, 2, 3, 4, 5**. En el objetivo, las escuchas con captura se procesan antes de las registradas sin captura. El `target` del clic sigue siendo el botón durante todo este recorrido.

No todos los eventos burbujean

`click`, `input`, `change` y `submit` sí. `focus`, `blur` y `mouseenter` no. Esta diferencia importa cuando quieras delegar una escucha.

06 preventDefault y stopPropagation

El navegador puede realizar una **acción predeterminada**: seguir un enlace, enviar un formulario o marcar una casilla. Esa acción y el recorrido del evento son cosas diferentes.

Método	Qué hace	Qué no hace
`preventDefault()`	Cancela la acción predeterminada de un evento cancelable.	No detiene su propagación.
`stopPropagation()`	Detiene la continuación del recorrido a otros elementos.	No cancela la acción predeterminada ni las otras escuchas del mismo elemento.
`stopImmediatePropagation()`	Detiene el recorrido y las escuchas que quedan del mismo elemento.	No cancela por sí mismo la acción predeterminada.

Ejemplo: un enlace que mostramos sin navegar

HTML

```html
<a id="enlace" href="#destino">Ir al destino</a>
<p id="aviso" role="status"></p>
<h2 id="destino">Destino</h2>
```

JavaScript · cancelar la navegación del enlace

```javascript
const enlace = document.querySelector("#enlace");
const aviso = document.querySelector("#aviso");

enlace.addEventListener("click", (evento) => {
  evento.preventDefault();
  aviso.textContent = "Has activado el enlace, sin navegar.";
});
```

Ejemplo: modificar el laboratorio anterior

Añade `evento.stopPropagation()` dentro del manejador del botón del apartado 05. Las escuchas de captura ya se han ejecutado; las de burbujeo en `zona` y `body` dejarán de ejecutarse.

JavaScript · sustituye el manejador del botón del apartado 05

```javascript
accion.addEventListener("click", (evento) => {
  console.log("3. botón: objetivo");
  evento.stopPropagation();
});
```

Cancelar con intención

Usa `preventDefault` cuando vayas a sustituir la acción del navegador. Evita detener la propagación por costumbre: podrías romper una escucha delegada en un padre.

Dos errores frecuentes

Con `addEventListener`, devolver `false` no cancela el evento. Y `preventDefault` no funciona si `cancelable` es falso o si la escucha se registró como `passive`.

07 Elegir el evento de interacción

No elijas un evento por costumbre: elígelo según la acción que necesitas detectar. Para activar una acción de un botón, `click` suele ser suficiente.

Evento	Cuándo se utiliza
`click`	Activación de un elemento, incluida la activación por teclado de controles nativos.
`dblclick`	Doble clic del dispositivo apuntador. No conviene que sea la única forma de realizar una acción.
`mousedown / mouseup`	Pulsación y liberación de un botón del ratón.
`mousemove`	Movimiento del ratón sobre un elemento; puede producir muchas llamadas.
`mouseenter / mouseleave`	Entrada o salida del conjunto formado por un elemento y sus descendientes. No burbujean.
`mouseover / mouseout`	Entrada o salida de un elemento; también se producen al pasar entre descendientes. Burbujean.
`pointerdown / pointerup / pointermove`	Familia común para ratón, entrada táctil y lápiz. Incluye datos como pointerType.

Observar unas coordenadas

HTML · zona de prueba

```html
<div id="panel" style="padding: 40px; border: 1px solid">
  Mueve el puntero por aquí
</div>
<p id="posicion"></p>
```

JavaScript · coordenadas relativas a la ventana visible

```javascript
const panel = document.querySelector("#panel");
const posicion = document.querySelector("#posicion");

panel.addEventListener("pointermove", (evento) => {
  posicion.textContent =
    `${evento.pointerType}: ${evento.clientX}, ${evento.clientY}`;
});
```

Propiedad	Sistema de referencia
`clientX / clientY`	Ventana visible del documento: viewport.
`pageX / pageY`	Página completa; incluye el desplazamiento por scroll.
`button`	Botón que ha cambiado de estado: 0 suele indicar el principal.

Una interfaz que también se pueda usar sin ratón

Usa botones, enlaces y etiquetas HTML adecuados. Un efecto al pasar el ratón puede complementar una acción, pero la acción debe poder realizarse también con teclado.

08 keydown, keyup y las teclas

`keydown` ocurre al presionar una tecla y puede repetirse si la mantienes pulsada. `keyup` ocurre al soltarla. Normalmente el evento se origina en el elemento que tiene el foco.

Propiedad	Ejemplo y significado
`key`	"a", "A", "Enter", "Escape" o "ArrowRight". Tiene en cuenta la distribución del teclado y modificadores.
`code`	"KeyA", "Enter" o "ArrowRight". Identifica la posición física de la tecla.
`ctrlKey / shiftKey / altKey / metaKey`	Booleanos que indican qué modificadores están pulsados.
`repeat`	true si este keydown es una repetición por mantener la tecla pulsada.
`isComposing`	Indica si el evento ocurre durante una composición de texto.

Ejemplo: limpiar una búsqueda con Escape

Escuchamos el teclado en el propio campo, en vez de capturar todas las teclas de la página.

HTML

```html
<label for="buscar">Buscar producto</label>
<input id="buscar" type="search">
<p id="resultado" role="status"></p>
```

JavaScript

```javascript
const buscar = document.querySelector("#buscar");
const resultado = document.querySelector("#resultado");

function actualizarBusqueda() {
  const texto = buscar.value.trim();
  resultado.textContent = texto ? `Buscando: ${texto}` : "Sin filtro";
}

buscar.addEventListener("input", actualizarBusqueda);
buscar.addEventListener("keydown", (evento) => {
  if (evento.key === "Escape" && !evento.isComposing) {
    buscar.value = "";
    actualizarBusqueda();
  }
});
```

Para leer texto, escucha input

El usuario también puede pegar, dictar o introducir texto con un sistema de composición. `keydown` sirve para teclas y atajos; `input` sirve para reaccionar al cambio del contenido. `keypress` es una API obsoleta: evita usarla.

Un cambio por JavaScript no dispara input por sí solo

En el ejemplo, después de asignar `buscar.value = ""`, llamamos a `actualizarBusqueda()`. Esa asignación no provoca automáticamente el evento `input`.

En atajos globales, comprueba si se está escribiendo en un input, textarea o elemento editable. Evita bloquear Tab, atajos del navegador o teclas que el control ya necesita.

09 input, change, focus y blur

En un formulario interesa saber si el contenido está cambiando, si se ha confirmado una selección o si un campo acaba de recibir o perder el foco.

Evento	Cuándo elegirlo
`input`	Actualizar una vista mientras el usuario modifica un valor: contador, filtro o previsualización.
`change`	Responder a un cambio confirmado: seleccionar una opción o marcar una casilla. En campos de texto, suele llegar al perder el foco si cambió el valor.
`focus / blur`	Detectar que un control gana o pierde el foco. No burbujean.
`focusin / focusout`	Detectar cambios de foco con burbujeo, por ejemplo desde el formulario padre.
`reset`	Detectar un restablecimiento del formulario. No sustituye al evento input.

Ejemplo: contador de caracteres y categoría

HTML

```html
<label for="nombre">Nombre del producto</label>
<input id="nombre" type="text" maxlength="30">
<p id="contador">0 / 30</p>

<label for="categoria">Categoría</label>
<select id="categoria">
  <option value="">Elige una categoría</option>
  <option value="hardware">Hardware</option>
  <option value="software">Software</option>
</select>
<p id="seleccion" role="status"></p>
```

JavaScript

```javascript
const nombre = document.querySelector("#nombre");
const contador = document.querySelector("#contador");
const categoria = document.querySelector("#categoria");
const seleccion = document.querySelector("#seleccion");

nombre.addEventListener("input", (evento) => {
  contador.textContent = `${evento.currentTarget.value.length} / 30`;
});

categoria.addEventListener("change", (evento) => {
  seleccion.textContent = `Categoría: ${evento.currentTarget.value}`;
});
```

Experimento para entender la diferencia

Añade temporalmente escuchas `input` y `change` al campo nombre. Escribe tres letras y después sal del campo con Tab. Observa cuándo aparece cada mensaje.

El foco forma parte de la interacción

El usuario puede recorrer los controles con Tab. Mantén una indicación visual del foco y mensajes legibles. Un error no debe comunicarse únicamente cambiando el color del borde.

10 La página también produce eventos

Los eventos no se limitan a pulsaciones. El documento, la ventana, una imagen o un reproductor multimedia pueden avisar de cambios en su estado.

Origen	Evento	Para qué sirve
`document`	`DOMContentLoaded`	El HTML se ha analizado y los scripts diferidos correspondientes se han ejecutado. No espera a todas las imágenes.
`window`	`load`	La página y los recursos que este evento espera han terminado de cargarse.
`window`	`resize`	Ha cambiado el tamaño de la ventana visible.
Una imagen	`load / error`	Su carga se ha completado o ha fallado.
Vídeo o audio	`loadedmetadata`	Los metadatos del medio están disponibles.
Vídeo o audio	`play / pause / ended`	Empieza la reproducción, se pausa o termina.
Vídeo o audio	`timeupdate`	Ha cambiado la posición de reproducción; no tiene una frecuencia fija.

Inicializar después de analizar el HTML

JavaScript · alternativa cuando no utilizas defer

```
document.addEventListener("DOMContentLoaded", () => {
  const boton = document.querySelector("#saludar");
  boton.addEventListener("click", () => {
    console.log("El botón ya existe");
  });
});
```

Elige una estrategia de carga

En los proyectos de esta guía utilizamos `<script src="app.js" defer>`. Con ese planteamiento puedes seleccionar elementos directamente en app.js; no necesitas envolver todo además en DOMContentLoaded.

10 Eventos multimedia

continuación

Un reproductor también cambia de estado. En este ejemplo escuchamos tres señales: reproducción, pausa y finalización. Los controles nativos del vídeo permiten probar cada acción sin escribir botones propios.

Ejemplo: informar del estado de un vídeo

HTML · añade un archivo de vídeo local a la carpeta

```html
<video id="video" controls src="demo.mp4"></video>
<p id="estado" role="status"></p>
```

JavaScript · escuchas en el propio reproductor

```javascript
const video = document.querySelector("#video");
const estado = document.querySelector("#estado");

video.addEventListener("play", () => {
  estado.textContent = "Reproduciendo";
});
video.addEventListener("pause", () => {
  estado.textContent = "En pausa";
});
video.addEventListener("ended", () => {
  estado.textContent = "Reproducción terminada";
});
```

Comprueba el origen y la propagación

Muchos eventos multimedia se escuchan directamente en el medio y no burbujean. Para probar este ejemplo necesitas un archivo `demo.mp4` válido en la misma carpeta.

11 Controles, etiquetas y nombres

Antes de escribir JavaScript, el formulario debe tener una estructura clara. Cada control necesita una etiqueta comprensible y un `name` si quieres recogerlo con `FormData`.

HTML · cuerpo del ejemplo para los apartados 11 y 12

```html
<form id="alta">
  <label for="nombre">Nombre</label>
  <input id="nombre" name="nombre" required>

  <label for="correo">Correo electrónico</label>
  <input id="correo" name="correo" type="email" required>

  <label for="edad">Edad</label>
  <input id="edad" name="edad" type="number" min="16" required>

  <label for="ciclo">Ciclo</label>
  <select id="ciclo" name="ciclo">
    <option value="daw">DAW</option>
    <option value="dam">DAM</option>
  </select>

  <fieldset>
    <legend>Intereses</legend>
    <label><input type="checkbox" name="intereses"
                  value="web"> Desarrollo web</label>
    <label><input type="checkbox" name="intereses"
                  value="datos"> Bases de datos</label>
  </fieldset>

  <label><input type="checkbox" name="acepta" required>
    Acepto las condiciones</label>
  <button type="submit">Registrar</button>
  <button type="reset">Restablecer</button>
</form>
```

Elemento o atributo	Función
`label for="…"`	Relaciona la etiqueta con el id del control; activarla permite enfocar o activar ese control.
`id`	Identifica un elemento del documento y permite seleccionarlo desde JS.
`name`	Es la clave con la que el control aparece en los datos del formulario.
`fieldset / legend`	Agrupan controles relacionados y dan un nombre al grupo.
`type="submit" / "reset" / "button"`	Envío, restablecimiento o botón sin una acción de formulario automática.

Declara el tipo de cada botón

Dentro de un formulario, un botón sin tipo suele actuar como submit. Para una acción auxiliar, como mostrar una contraseña, escribe `type="button"`.

12 value, checked y FormData

Puedes leer un control directamente o recoger el conjunto de datos con `FormData`. Los valores de texto y número se obtienen normalmente como **cadenas**; una casilla tiene además el booleano `checked`.

JavaScript · lectura directa sobre el HTML anterior

```
const edad = document.querySelector("#edad");
const acepta = document.querySelector('[name="acepta"]');

console.log(edad.value);   // Por ejemplo: "18"
console.log(acepta.value); // "on", aunque no esté marcada
console.log(acepta.checked); // true o false

const edadNumerica = edad.valueAsNumber;
// Un campo numérico vacío produce NaN en valueAsNumber.
```

Leer todos los controles con nombre

JavaScript · añade este código a app.js con defer

```
const formulario = document.querySelector("#alta");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const datos = new FormData(evento.currentTarget);

  const nombre = String(datos.get("nombre") ?? "").trim();
  const edadTexto = String(datos.get("edad") ?? "");
  const edad = edadTexto === "" ? NaN : Number(edadTexto);
  const intereses = datos.getAll("intereses");
  const acepta = datos.has("acepta");

  console.log({ nombre, edad, intereses, acepta });
});
```

Operación	Qué devuelve o representa
`datos.get("ciclo")`	El primer valor de esa clave; null si no existe.
`datos.getAll("intereses")`	Un array con los valores seleccionados de esa clave.
`datos.has("acepta")`	Si existe la clave: en esta casilla, indica que está marcada.
`for (const [clave, valor] of datos)`	Recorre las entradas, incluidas las claves repetidas.

Lo que FormData recoge y lo que omite

Recoge controles que participan en el envío: con `name`, habilitados y, en casillas y radios, seleccionados. Un campo sin name o con `disabled` no aparece. Los archivos se representan mediante objetos `File`. Construir FormData no valida los campos ni envía los datos.

Dos conversiones que pueden engañarte

`Number("")` devuelve 0: comprueba antes el vacío. `Object.fromEntries(datos)` no conserva todas las entradas si una clave se repite; para las casillas de intereses necesitas `getAll`.

13 El evento submit pertenece al formulario

Escucha `submit` en el formulario cuando quieras recoger sus datos. Así manejas el envío iniciado por el botón y el envío implícito, por ejemplo con Enter cuando la estructura del formulario lo permite.

HTML · ejemplo independiente

```html
<form id="contacto">
  <label for="email">Correo electrónico</label>
  <input id="email" name="email" type="email" required>
  <button type="submit">Guardar</button>
</form>
<p id="respuesta" role="status"></p>
```

JavaScript · procesamiento local, sin recargar la página

```javascript
const contacto = document.querySelector("#contacto");
const respuesta = document.querySelector("#respuesta");

contacto.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const datos = new FormData(evento.currentTarget);
  respuesta.textContent = `Correo recogido: ${datos.get("email")}`;
});
```

¿Qué ocurre antes del submit?

Intento de envío

El usuario activa el botón.

Validación HTML

Si es inválido, el navegador muestra el problema y no llega a submit.

submit

Si es válido, se ejecuta el manejador. Aquí podemos cancelar el envío y procesar los datos.

Si el listener no se ejecuta, revisa required y type

Con la validación automática activa, un email incorrecto o un campo obligatorio vacío pueden impedir que se dispare `submit`. No es necesariamente un fallo del registro de la escucha.

Acción	Comportamiento relevante
`formulario.requestSubmit()`	Inicia el proceso de envío con validación y submit, salvo que se omita la validación según los atributos del formulario o botón.
`formulario.submit()`	Envía directamente; omite la validación y no dispara el evento submit.
`novalidate` en el formulario	Desactiva la comprobación automática al enviar; JavaScript aún puede llamar a reportValidity.

El botón que ha provocado el envío

En un `SubmitEvent`, `evento.submitter` identifica el botón que ha iniciado el envío, si lo hay. Si sus `name` y `value` importan, puedes usar `new FormData(formulario, evento.submitter)`.

En estos ejemplos no hay un servidor: `preventDefault` evita el envío tradicional y permite mostrar el resultado en la propia página.

14 Primero expresa las reglas en el HTML

Validar consiste en comprobar que los datos cumplen unas reglas. Antes de escribir condiciones manuales, utiliza las restricciones que el navegador ya entiende.

Atributo	Regla	Ejemplo
`required`	Debe existir un valor. En una casilla, debe estar marcada.	`<input required>`
`type="email"`	Formato de correo según las reglas de HTML.	`<input type="email">`
`minlength / maxlength`	Longitud del texto.	`minlength="3" maxlength="30"`
`min / max`	Límites de un valor numérico o de otros tipos que los admiten.	`min="1" max="100"`
`step`	Incrementos admitidos en un campo numérico.	`step="0.01"`
`pattern`	Formato del valor mediante un patrón.	`pattern="DAW-\d{3}"`

HTML · precio no negativo con dos decimales

```html
<label for="precio">Precio en euros</label>
<input id="precio" name="precio" type="number"
       min="0" step="0.01" required>
```

Consultar el estado de validación

Cada control tiene una propiedad `validity`. Sus indicadores permiten saber por qué no es válido.

Indicador	Causa
`valueMissing`	Falta un valor obligatorio.
`typeMismatch / patternMismatch`	No cumple el formato del tipo o el patrón.
`tooShort / tooLong`	La longitud no cumple las restricciones aplicables.
`rangeUnderflow / rangeOverflow`	El valor está por debajo del mínimo o por encima del máximo.
`stepMismatch / badInput`	No cumple el incremento o el navegador no puede convertir la entrada.
`customError / valid`	Hay un error personalizado / se cumplen todas las restricciones.

JavaScript · consultar y mostrar el resultado

```javascript
const precio = document.querySelector("#precio");

console.log(precio.checkValidity()); // true o false
console.log(precio.validity.rangeUnderflow);
precio.reportValidity(); // Comprueba y muestra el aviso si es inválido.
```

checkValidity y reportValidity

Ambos devuelven un booleano y pueden disparar `invalid` en controles inválidos. `reportValidity` añade la presentación del problema al usuario. El evento `invalid` no burbujea; desde un formulario puedes escucharlo en captura.

El navegador comprueba formato, no toda la lógica

`required` no elimina espacios, ni `type="email"` prueba que una dirección exista. Las reglas de negocio, como no admitir nombres formados solo por espacios, necesitan validación adicional.

15 Patrones para reconocer texto

Una **expresión regular** describe un patrón de caracteres. Úsala para comprobar formatos concretos, no para sustituir todas las reglas del formulario.

JavaScript · código DAW seguido de tres dígitos

```
const patron = /^DAW-\d{3}$/;

console.log(patron.test("DAW-042"));  // true
console.log(patron.test("DAW-42"));   // false
console.log(patron.test("XDAW-042")); // false
console.log(patron.test("daw-042"));  // false
```

Símbolo	Significado en este patrón
`^`	Inicio del texto.
`DAW-`	Los caracteres literales D, A, W y guion.
`\d`	Un dígito del 0 al 9.
`{3}`	Exactamente tres repeticiones del elemento anterior.
`$`	Final del texto.
`.test(texto)`	Devuelve true si encuentra una coincidencia.

Otros patrones pequeños

Patrón JavaScript	Qué comprueba
`/^\d+$/`	Uno o más dígitos, sin ningún otro carácter.
`/^[A-Z]{2}\d{4}$/`	Dos letras mayúsculas ASCII seguidas de cuatro dígitos.
`/^DAW-\d{3}$/i`	El patrón anterior ignorando mayúsculas y minúsculas.

El mismo formato en un atributo pattern

HTML · patrón sin las barras delimitadoras de JavaScript

```
<label for="codigo">Código de alumno, por ejemplo DAW-042</label>
<input id="codigo" name="codigo" pattern="DAW-\d{3}" required>
<p>Escribe DAW- seguido de tres dígitos.</p>
```

pattern debe coincidir con el valor completo

En HTML no se escriben las barras `/…/`, ni hace falta añadir `^` y `$` para exigir todo el valor. Sin `required`, un campo vacío puede ser válido. Los patrones actuales de HTML usan reglas Unicode con la bandera `v`.

Tres decisiones prácticas

Usa `trim()` antes de comprobar un identificador si la regla permite ignorar espacios exteriores. Evita la bandera `g` en validaciones repetidas con `test`, porque mantiene un índice interno. Para correos, empieza por `type="email"`; no inventes un patrón enorme.

16 Añadir una regla propia con setCustomValidity

Algunas reglas dependen de varios controles. Por ejemplo, dos códigos escritos por el usuario deben coincidir. `setCustomValidity` integra tu regla en el sistema de validación del navegador.

HTML · ejemplo independiente

```html
<form id="confirmacion" novalidate>
  <label for="codigo1">Código de alumno</label>
  <input id="codigo1" name="codigo1" required>
  <label for="codigo2">Repite el código</label>
  <input id="codigo2" name="codigo2" required>
  <button type="submit">Comprobar</button>
</form>
<p id="validado" role="status"></p>
```

JavaScript · comprobar, informar y limpiar el error

```javascript
const form = document.querySelector("#confirmacion");
const codigo1 = document.querySelector("#codigo1");
const codigo2 = document.querySelector("#codigo2");
const validado = document.querySelector("#validado");

function validarCoincidencia() {
  codigo2.setCustomValidity(""); // Borra el error anterior.
  if (codigo1.value !== codigo2.value) {
    codigo2.setCustomValidity("Los dos códigos deben coincidir.");
  }
}

form.addEventListener("input", () => {
  validarCoincidencia();
  validado.textContent = "";
});

form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  validarCoincidencia();
  if (!form.reportValidity()) return;
  validado.textContent = "Los datos cumplen las reglas.";
});
```

1. `novalidate` permite llegar al manejador de submit incluso con campos inválidos.
2. Primero se recalcula la regla personalizada y se borra el posible error anterior.
3. `reportValidity()` comprueba también los required del HTML.
4. Solo se continúa si el formulario completo es válido.

Una cadena vacía es imprescindible

`setCustomValidity("Mensaje")` marca el campo como inválido. `setCustomValidity("")` elimina ese error personalizado. Si no lo borras, el campo puede seguir siendo inválido aunque el usuario corrija el valor.

Validar también en el servidor

La validación en cliente mejora la interacción, pero el usuario puede modificar el código o enviar datos por otra vía. Si se guardan datos en un servidor, el servidor debe aplicar sus propias comprobaciones.

17 Delegación de eventos

Si una lista crece, no necesitas registrar una escucha nueva en cada botón. Puedes escuchar en un **contenedor estable** y averiguar qué botón originó el clic. Aprovechas el burbujeo del evento.

HTML · botones dentro de una lista

```html
<ul id="lista">
  <li>Teclado
    <button type="button" data-accion="borrar">
      <span>Eliminar</span>
    </button>
  </li>
  <li>Ratón
    <button type="button" data-accion="borrar">Eliminar</button>
  </li>
</ul>
```

JavaScript · una escucha para toda la lista

```javascript
const lista = document.querySelector("#lista");

lista.addEventListener("click", (evento) => {
  if (!(evento.target instanceof Element)) return;

  const boton = evento.target.closest('button[data-accion="borrar"]');
  if (!boton || !lista.contains(boton)) return;

  const fila = boton.closest("li");
  if (!fila || !lista.contains(fila)) return;

  fila.remove();
});
```

¿Por qué se utiliza closest?

Si pulsas sobre el `span`, `target` es ese span. `closest` busca desde el propio elemento hacia sus antecesores y localiza el botón que coincide con el selector.

Paso	Qué resuelve
`target instanceof Element`	Asegura que se pueden usar métodos de Element sobre el objetivo.
`closest("button…")`	Identifica el botón aunque se pulse un elemento interior.
`lista.contains(boton)`	Limita la acción al contenedor que gestiona esta escucha.
`boton.closest("li")`	Obtiene la fila a la que pertenece el botón.
`data-accion`	Guarda un dato semántico propio en el HTML; se lee como dataset.accion.

También funciona con nuevos botones

Si después creas y añades otro li con su botón, su clic llegará a la escucha de `lista`. La escucha permanece mientras el contenedor siga siendo el mismo.

Una precaución al renderizar

Si sustituyes el contenedor completo, la escucha del contenedor antiguo no pasa al nuevo. Conserva el ul y renueva sus hijos. Para una aplicación con un modelo de datos, actualiza el modelo y vuelve a renderizar; eso se hace en el proyecto siguiente.

18 Un catálogo de productos

Vamos a unir formulario, validación, modelo y delegación. La aplicación añadirá productos a un array, representará ese array en una lista y permitirá eliminarlos. No utiliza servidor ni guarda datos después de recargar.

submit → **Validar y leer** → **Actualizar el array** → **Renderizar el DOM**

HTML · index.html completo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Catálogo de productos</title>
  <script src="app.js" defer></script>
</head>
<body>
  <h1>Catálogo de productos</h1>
  <form id="formProducto" novalidate>
    <label for="nombre">Nombre del producto</label>
    <input id="nombre" name="nombre" maxlength="40" required>

    <label for="precio">Precio en euros</label>
    <input id="precio" name="precio" type="number"
           min="0" step="0.01" required>

    <button type="submit">Añadir</button>
  </form>

  <p id="mensaje" role="status"></p>
  <p id="vacio">No hay productos.</p>
  <ul id="productos" aria-label="Productos del catálogo"></ul>
</body>
</html>
```

Cómo montar el proyecto

Crea una carpeta nueva y guarda este HTML como `index.html`. En `app.js`, copia **las dos partes de JavaScript de las páginas siguientes, en ese orden**. Son un único archivo, no dos ejemplos independientes.

Qué reglas se van a aplicar

El nombre, una vez quitados los espacios exteriores, tendrá al menos tres caracteres. El precio será un número no negativo y respetará los incrementos de 0,01 indicados en el HTML. Los mensajes se muestran sin recargar la página.

Prueba prevista: añadir un teclado de 29,90 €, añadir un ratón, eliminar el teclado y eliminar el ratón. La lista vacía debe volver a mostrar «No hay productos».

18 Modelo, validación y alta

continuación

El array es el **modelo**. El DOM es su representación. El manejador de submit coordina la lectura, la validación, el cambio del modelo y la actualización de la interfaz.

JavaScript · app.js · parte 1 de 2

```
const form = document.querySelector("#formProducto");
const nombreInput = document.querySelector("#nombre");
const precioInput = document.querySelector("#precio");
const lista = document.querySelector("#productos");
const mensaje = document.querySelector("#mensaje");
const vacio = document.querySelector("#vacio");

let productos = [];
let siguienteId = 1;

function leerProducto() {
  const datos = new FormData(form);
  return {
    nombre: String(datos.get("nombre") ?? "").trim(),
    precio: Number(datos.get("precio"))
  };
}

function validarFormulario() {
  nombreInput.setCustomValidity("");
  precioInput.setCustomValidity("");
  if (nombreInput.value.trim().length < 3) {
    nombreInput.setCustomValidity("Escribe al menos 3 caracteres.");
  }
  if (precioInput.value !== "" &&
      !Number.isFinite(precioInput.valueAsNumber)) {
    precioInput.setCustomValidity("Escribe un número válido.");
  }
  return form.reportValidity();
}

form.addEventListener("input", () => {
  nombreInput.setCustomValidity("");
  precioInput.setCustomValidity("");
  mensaje.textContent = "";
});

form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  mensaje.textContent = "";
  if (!validarFormulario()) return;

  const producto = { id: siguienteId++, ...leerProducto() };
  productos.push(producto);
  renderizarProductos();
  form.reset();
  nombreInput.focus();
  mensaje.textContent = `Añadido: ${producto.nombre}.`;
});
```

Por qué convertimos después de validar

`required` impide continuar con un precio vacío. Solo cuando `validarFormulario()` devuelve true ejecutamos `leerProducto()`. Así evitamos interpretar el vacío como 0.

`novalidate` deja que la comprobación se coordine desde submit; `reportValidity` sigue aplicando los límites HTML y los errores personalizados. La función de renderizado se define en la parte 2.

18 Renderizado y escucha delegada

continuación

Los identificadores permiten reconocer cada producto sin depender de su posición en el array. Al eliminar uno, primero cambiamos los datos y después reconstruimos los hijos de la lista.

JavaScript · app.js · parte 2 de 2 · continuación

```
function renderizarProductos() {
  lista.replaceChildren();
  vacio.hidden = productos.length > 0;

  for (const producto of productos) {
    const fila = document.createElement("li");
    const texto = document.createElement("span");
    texto.textContent =
      `${producto.nombre} - ${producto.precio.toFixed(2)} € `;

    const boton = document.createElement("button");
    boton.type = "button";
    boton.dataset.accion = "borrar";
    boton.dataset.id = String(producto.id);
    boton.setAttribute("aria-label", `Eliminar ${producto.nombre}`);

    const etiqueta = document.createElement("span");
    etiqueta.textContent = "Eliminar";
    boton.append(etiqueta);
    fila.append(texto, boton);
    lista.append(fila);
  }
}

lista.addEventListener("click", (evento) => {
  if (!(evento.target instanceof Element)) return;
  const boton = evento.target.closest('button[data-accion="borrar"]');
  if (!boton || !lista.contains(boton)) return;

  const id = Number(boton.dataset.id);
  const producto = productos.find((item) => item.id === id);
  if (!producto) return;

  productos = productos.filter((item) => item.id !== id);
  renderizarProductos();
  nombreInput.focus();
  mensaje.textContent = `Eliminado: ${producto.nombre}.`;
});

renderizarProductos();
```

Decisión	Qué aporta
`replaceChildren()`	Vacía los hijos del ul; conserva el propio ul y su escucha.
`textContent`	Muestra el nombre como texto, sin interpretarlo como HTML.
`dataset.id`	Guarda un identificador como texto; se convierte a número al utilizarlo.
`closest`	La eliminación funciona al pulsar el span interior del botón.
`role="status"` y foco	Anuncia los cambios y proporciona un destino de foco después de modificar la lista.

Comprueba el modelo y la vista

Si solo borrases un li, el producto seguiría en el array y reaparecería al renderizar. Si solo cambiases el array, el DOM conservaría una vista desactualizada. Ambas partes deben mantenerse coherentes.

19 Cuando una interacción no funciona

No cambies código al azar. Recorre el flujo de la interacción: elemento, registro de la escucha, evento, datos, validación y actualización.

Síntoma	Qué revisar
Error al llamar a addEventListener	¿querySelector devuelve null? Comprueba el id, el selector y cuándo se carga el script.
La función se ejecuta al cargar	¿Has escrito manejador() en lugar de manejador al registrarla?
Una acción ocurre varias veces	¿Registras nuevas funciones dentro de renderizar, submit o de otra función que se repite?
El listener del formulario no se ejecuta	¿La validación HTML está deteniendo el envío antes del submit?
La página se recarga al enviar	¿Cancelas el evento submit con preventDefault? ¿El botón auxiliar es type="button"?
Un botón nuevo no funciona	¿Solo registraste escuchas en botones que existían al inicio? ¿Puedes delegar en un padre estable?
Pulsar el texto interior no funciona	¿Compruebas target directamente sin utilizar closest para encontrar el botón?
El campo sigue inválido después de corregirlo	¿Has limpiado el error con setCustomValidity("")?
Un padre deja de recibir clics	¿Hay stopPropagation en algún descendiente? ¿Has sustituido el contenedor?

Hacer visible la información del evento

JavaScript · usa esta traza dentro de un manejador

```
console.log("tipo:", evento.type);
console.log("origen:", evento.target);
console.log("escucha:", evento.currentTarget);
console.log("fase:", evento.eventPhase);
console.log("cancelado:", evento.defaultPrevented);
// eventPhase: 1 captura, 2 objetivo, 3 burbujeo; 0 fuera del recorrido.
```

Un procedimiento de depuración

1. Reproduce el fallo con la misma acción.
2. Comprueba en Elements que existe el elemento esperado.
3. Pon un breakpoint en la primera línea del manejador.
4. Si no se detiene, revisa registro, tipo de evento y validación.
5. Si se detiene, inspecciona valores y avanza instrucción a instrucción.
6. Comprueba el array y el DOM después de la acción.

Manejadores pequeños

Un manejador debería coordinar tareas con nombres claros: leer, validar, actualizar y renderizar. Separa las reglas de datos del código que solo muestra la interfaz; así podrás explicar y depurar cada parte.

20 Qué utilizar en cada situación

Quiero…	Utilizo…
Responder a una activación	`elemento.addEventListener("click", manejador)`
Retirar una escucha	`removeEventListener` con la misma función y captura
Leer el origen o la escucha actual	`evento.target` / `evento.currentTarget`
Evitar la acción del navegador	`evento.preventDefault()`
Detener el recorrido del evento	`evento.stopPropagation()`, cuando esté justificado
Responder a cambios mientras se escribe	`input`
Responder a una selección confirmada	`change`
Detectar una tecla concreta	`keydown` y `evento.key`
Procesar el formulario	`submit` en el propio formulario
Recoger los valores con nombre	`new FormData(formulario)`
Recoger varias casillas con la misma clave	`datos.getAll("clave")`
Comprobar reglas HTML y mostrar errores	`formulario.reportValidity()`
Aplicar o borrar un error personalizado	`setCustomValidity("Mensaje")` / `setCustomValidity("")`
Reconocer un formato de texto	`patron.test(texto)` o `pattern` en HTML
Gestionar botones creados dinámicamente	Escucha en el padre + `target.closest(…)`

La idea que conecta todo el tema

La interacción inicia un evento. El manejador decide qué hacer. Si hay datos, primero los valida; después actualiza el modelo y finalmente la interfaz. El DOM y el modelo deben describir el mismo estado.

Documentación para consultar

Los ejemplos y las explicaciones son didácticos. Para consultar los detalles de las APIs, utiliza estas referencias de MDN Web Docs y del estándar DOM. Las consultas no requieren conexión para leer esta guía; solo los enlaces externos necesitan Internet.

Escuchas y propagación

- addEventListener
- removeEventListener
- Propagación y delegación
- preventDefault
- stopPropagation
- DOM Standard: recorrido del evento

Controles y datos

- KeyboardEvent.key
- Evento input
- Evento change
- FormData
- Evento submit
- Eventos multimedia

Validación

- Validación de restricciones
- reportValidity
- setCustomValidity
- Atributo pattern
- Expresiones regulares
- Eventos de puntero